Pandas Practice Sheet – Day 2

Covers everything from today's notebook (pandas_2): string filtering, new columns, unique/nunique, nulls, duplicates, statistics, apply, datetime, groupby.

How to use: copy the setup code into a notebook, solve every question in your own cell, then show me your notebook. Write the code yourself (no looking at the notes first). Total: 53 questions.

Setup – run this first

```
import pandas as pd
import numpy as np

# Dataset A – employees
emp = pd.DataFrame({
    "Name": ["Alice", "Bob", "Charlie", "David", "Eve", "Frank", "Grace",
"Hannah", "Sakib", "Umar"],
    "City": ["New York", "Los Angeles", "Newark", "Boston", "New Delhi",
"Chicago", "New Orleans", "Houston", "H Los Ang", "Austin"],
    "Department": ["HR", "IT", "Finance", "IT", "HR", "Marketing", "Finance",
"HR", "HR", "IT"],
    "Salary": [50000, 60000, 55000, 70000, 52000, 58000, 62000, 51000, 70000,
65000]
})

# Dataset B – students (has NaN, a duplicate row, and dates as text)
stu = pd.DataFrame({
    "StudentID": [101, 102, 103, 104, 105, 106, 107, 101, 108, 109],
    "FullName": ["Rahim Uddin", "Karim Hasan", "Afsana Mimi", "Sadia
Chowdhury", "Alif Rahman",
                 "Tanvir Ahmed", "Nusrat Jahan", "Rahim Uddin", None, "Imran
Hossain"],
    "Data Structure Marks": [92, 85, 78, 95, 88, np.nan, 70, 92, 81, 67],
    "Python Marks": [88, 90, np.nan, 85, 92, 79, 83, 88, 75, np.nan],
    "Algorithm Marks": [90, 80, 85, np.nan, 87, 76, 91, 90, 70, 82],
    "Instructor": ["Sakib", "Nabila", "Sakib", "Nabila", "Sakib", "Tahmid",
"Tahmid", "Sakib", "Nabila", "Tahmid"],
    "EnrollmentDate": ["2024-01-10", "2024-01-15", "2024-02-01", "2024-02-
10", "2024-03-05",
                       "2024-03-12", "2024-04-01", "2024-01-10", "2024-04-
```

```
20", "2024-05-02"],
    "FinishedDate": ["2024-06-10", "2024-07-01", "2024-07-15", "2024-08-01",
"2024-08-20",
                     "2024-09-05", "2024-09-30", "2024-06-10", "2024-10-15",
"2024-11-01"]
})
```

Section 1 – String filtering with str.contains (use `emp`)

1. Show all rows where `City` contains the text "New".
2. Show all rows where `City` contains "new" in any letter case.
3. Show all rows where `City` starts with "New".
4. Show all rows where `City` ends with "s".
5. Show all rows where `Name` starts with a vowel (A, E, I, O, U).
6. Show all rows where `City` contains "Los" or "Aus".
7. Show rows where `City` contains "New" AND `Salary` is above 52000.
8. Show rows where `City` does NOT contain "New".

Section 2 – Adding new columns (use `stu`)

9. Add a column `Country` with the constant value "Bangladesh".
10. Add `Total Marks` = Data Structure + Python + Algorithm marks (just the `+` version). Look at the result: what happens to rows with NaN?
11. Using `np.where`, add `DS Grade`: "A+" if Data Structure Marks is above 90, otherwise "A".
12. Add a True/False column `Passed in DS` that is True when Data Structure Marks is above 70.
13. Add `First Name` by splitting `FullName` on the space and taking the first part.
14. Add `Last Name` the same way (second part).
15. Save the `stu` DataFrame to a file called `stu_new.csv`, then read it back into `stu_check`. Why do you see an extra `Unnamed: 0` column, and how can you avoid it?

Section 3 – unique and nunique

16. Print the unique values of `Instructor`.

17. How many different instructors are there? Do it two ways: with `len()` and with `nunique()`.
18. `unique()` and `nunique()` give different answers on `Data Structure Marks`. Show both and explain the difference in one line (comment).
19. Get the unique count of every column in `stu` in a single line.

Section 4 – Checking nulls

20. Show a True/False DataFrame of missing values for the whole `stu`.
21. Count the missing values in each column.
22. Show only the rows where `Python Marks` is null.
23. Show only the rows where `Python Marks` is NOT null.
24. Check with `hasnans` whether `StudentID` and `Python Marks` contain any NaN.

Section 5 – Duplicates

25. Show a True/False Series marking duplicate rows in `stu`, then count how many duplicates exist.
26. Show the duplicate rows themselves.
27. Remove full-row duplicates without changing `stu` permanently, then do it permanently with `inplace=True`.
28. Using the original `stu` (re-run setup), drop duplicates based only on `StudentID`, keeping the last occurrence.
29. Using the original `stu`, drop duplicates based on both `StudentID` and `FullName`.

Section 6 – Handling null values (use `stu` after removing duplicates)

30. Drop every row that has at least one NaN.
31. Drop rows only where ALL values are NaN. Will anything be removed here? Explain in a comment.
32. Drop rows where `Python Marks` OR `Algorithm Marks` is NaN.
33. Fill the null values in `FullName` with "unknown".
34. Fill the null values in `Python Marks` with the mean of that column.

35. Why is `df.fillna(0)` a poor choice for the whole of `stu`? Answer in a comment and name one column where it is acceptable.

Section 7 – Statistical functions

36. For `Data Structure Marks` find the sum, min, max, mean, median, mode and standard deviation.
37. Find the correlation between `Data Structure Marks` and `Python Marks`.
38. Find the column-wise sum, then the row-wise sum, of the three marks columns.
39. Create `Total Marks` using `iloc` and `sum(axis=1)` on the three marks columns, then run `describe()` on the DataFrame. Compare this total with your Q10 result and explain the difference.

Section 8 – apply()

40. Create `Scaled Marks` using min-max scaling on `Total Marks` with `apply` and a lambda: (x − min) / (max − min).
41. Write a function `grading_system(marks)` returning "A+" for 260 or more, "A" for 250 or more, otherwise "A-". Apply it to `Total Marks` into a column `Grade`.
42. Write a row-wise function that returns DS × 2 + Python × 3 + Algorithm × 4 and apply it with `axis=1` into `Weighted Marks`.

Section 9 – Datetime and timedelta

43. Check the dtype of `EnrollmentDate`, convert both date columns with `pd.to_datetime`, and check the dtype again.
44. Add `Enrollment Year`, `Enrollment Month` and `Enrollment Day` columns using `.dt`.
45. Add `Days Taken` = FinishedDate − EnrollmentDate. Then extract only the number of days as an integer column.
46. Show the students who took more than 150 days to finish.

Section 10 – GroupBy

47. Create a groupby object on `Instructor` and loop over it, printing each instructor name and that group's DataFrame.

48. Find the mean `Total Marks` per instructor.
49. Find the min and max of `Python Marks` per instructor.
50. Run `describe()` on the instructor groups.
51. Set `Total Marks` to NaN for two students of your choice. Then fill those NaN values with the mean `Total Marks` of the same instructor, using `map()` and `fillna()`.
52. Using `emp`, find the average salary per `Department`, then show which department has the highest average.

Bonus challenge

53. In one chain of operations on `stu`: remove duplicates, convert `EnrollmentDate` to datetime, keep only students whose `Python Marks` is not null, and show the result sorted by `Total Marks` (highest first).

When you finish, show me your notebook and I will check it, tell you which topics you practised well, and point out any that need more work.